

0/1 Sırt Çantası Probleminin Dueling Double DQN ve Önceliklendirilmiş Deneyim Tekrarı ile Çözümü: Ödül Şekillendirme Yaklaşımı ve Kapsamlı Performans Analizi

Mert EVRAN, Sefa ÖZKAN, Nihat KARAKUZU

Yazılım Mühendisliği Bölümü

Hasan Ferdi Turgutlu Teknoloji Fakültesi, Manisa Celal Bayar Üniversitesi

Email: {mertevran1907, ozkansefa2005, nihay64484}@gmail.com

Abstract—0/1 Sırt Çantası Problemi (Knapsack Problem – KP), kaynak kısıtı altında maksimum faydayı sağlayacak eşya alt kümesinin seçilmesini gerektiren NP-Zor bir kombinatoriyal optimizasyon problemidir. Bu çalışmada, problemi bir Markov Karar Süreci (MDP) olarak modelleyerek çözen gelişmiş bir Derin Pekiştirmeli Öğrenme (DRL) ajanı sunulmaktadır. Ajan; Dueling Double DQN mimarisi ve Önceliklendirilmiş Deneyim Tekrarı (PER) ile donatılmış, ayrıca Müfredat Öğrenme (Curriculum Learning) stratejisiyle eğitilmiştir. $N = 1000$ boyutuna kadar genişletilen test senaryolarında, geliştirilen ajanın Genetik Algoritma (GA) ile rekabetçi sonuçlar ürettiği ve asimptotik çıkarım süresini (~ 4.75 ms) $O(1)$ seviyesinde sabit tutarak geleneksel yöntemlere karşı üstün bir ölçeklenebilirlik sergilediği kanıtlanmıştır.

Index Terms—0/1 Sırt Çantası Problemi, Derin Pekiştirmeli Öğrenme, Dueling Double DQN, Önceliklendirilmiş Deneyim Tekrarı, Ödül Şekillendirme, Algoritma Analizi.

Abstract—The 0/1 Knapsack Problem (KP) is an NP-Hard combinatorial optimization problem that requires selecting a subset of items to maximize utility under a resource constraint. In this study, an advanced Deep Reinforcement Learning (DRL) agent is presented, which solves the problem by modeling it as a Markov Decision Process (MDP). The agent is equipped with the Dueling Double DQN architecture and Prioritized Experience Replay (PER), and it is trained using a Curriculum Learning strategy. In test scenarios scaled up to $N = 1000$, it has been proven that the developed agent produces competitive results with Genetic Algorithms (GA) and exhibits superior scalability against traditional methods by maintaining an asymptotic inference time (~ 4.75 ms) at a constant $O(1)$ level.

Index Terms—0/1 Knapsack Problem, Deep Reinforcement Learning, Dueling Double DQN, Prioritized Experience Replay, Reward Shaping, Algorithm Analysis.

I. GİRİŞ

Sırt çantası problemi (Knapsack Problem - KP), kombinatoriyal optimizasyonun en temel ve klasik problemlerinden biri olup, ilk olarak 1897 yılında matematikçi Tobias Dantzig tarafından formüle edilmiştir. Günümüzde lojistik planlama, finansal portföy yönetimi, kaynak tahsisi ve kargo yükleme gibi operasyonel süreçlerden kriptografiye kadar geniş bir uygulama yelpazesinde karşımıza çıkmaktadır. Problemin temel matematiksel formülasyonu şu şekildedir:

$$\begin{aligned} \max \quad & \sum_{j=1}^n v_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n w_j x_j \leq C \\ & x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\} \end{aligned}$$

Burada n toplam eşya sayısını, v_j her bir eşyanın değerini (faydasını), w_j eşyanın ağırlığını, C ise sırt çantasının maksimum taşıma kapasitesini temsil etmektedir.

0/1 KP'nin NP-Zor sınıfında yer alması, eşya sayısı (n) arttıkça olası çözüm uzayının 2^n şeklinde eksponansiyel büyümesine neden olmaktadır. Dinamik Programlama (DP) ve Dal-Sınır (Branch-and-Bound) gibi kesin çözüm yöntemleri küçük ve orta ölçekli problemler için optimum sonuç üretse de, yüksek boyutlu (örneğin $n \geq 1000$) durumlarda hesaplama süresi ve bellek maliyeti açısından pratikte kullanılamaz hale gelmektedir. Bu kısıtları aşmak için geliştirilen açgözlü (Greedy) yaklaşımlar ve Genetik Algoritma (GA) gibi meta-sezgiseller ise problem özelinde el ile parametre ayarı (fine-tuning) gerektirmekte ve genelleme yetenekleri noktasında sınırlı kalmaktadır.

Son yıllarda, **derin pekiştirmeli öğrenme (Deep Reinforcement Learning - DRL)** yöntemleri, özellikle Deep Q-Networks (DQN) [8] ile birlikte optimizasyon problemlerinde çığır açan bir yaklaşım sunmuştur. DRL'nin en büyük avantajı, ajanın deneme-yanılma yoluyla veri içerisindeki karmaşık örüntüleri (state-action mappings) kendi kendine öğrenebilmesi ve eğitim aşamasından sonra yeni problem örnekleri için neredeyse sabit sürede ($O(1)$ veya $O(N \cdot L)$ karmaşıklığında) çıkarım yapabilmesidir.

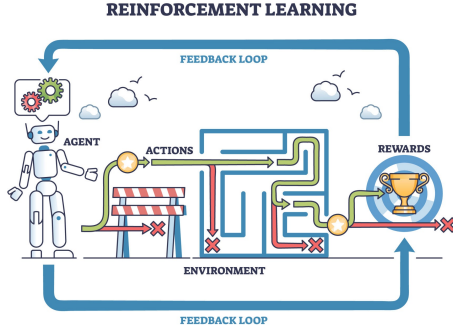


Fig. 1: Pekiştirmeli Öğrenme (RL) temel döngüsü [15].

Bu çalışmanın temel katkıları şu şekilde sıralanabilir:

- 1) 0/1 KP için özgün bir Markov Karar Süreci (MDP) ortamı tasarlanmış ve ajan-çevre etkileşimi Knapsack kurallarına göre optimize edilmiştir.
- 2) Dueling Double DQN mimarisi ve Önceliklendirilmiş Deneyim Tekrarı (PER) ile donatılmış, kararlı yakınsama sağlayan bir model gerçekleştirilmiştir.
- 3) Eşyaların değer/ağırlık verimliliğini ve kapasite doluluk oranını maksimize eden özgün bir **ödül şekillendirme (reward shaping)** stratejisi geliştirilmiştir.
- 4) Akademik benchmark setleri ve $N = 1000$ boyutuna varan yüksek karmaşıklıkla sentetik veriler üzerinde DP, Greedy ve GA ile kapsamlı bir karşılaştırmalı analiz sunulmuştur.
- 5) Yapılan deneylerde, eğitilen ajanın yüksek boyutlu problemlerde meta-sezgisel yöntemlerle (GA) rekabetçi skorlar üretebildiği, müfredat öğrenme (curriculum learning) stratejisi sayesinde karmaşık senaryolara adapte olduğu ve karar verme süresini problem boyutundan bağımsız olarak asimptotik olarak $O(1)$ seviyesinde sabitlediği deneysel olarak doğrulanmıştır.

II. LİTERATÜR TARAMASI

Sırt Çantası Problemi (KP) literatürü, 1960'lı yıllardan bu yana oldukça zengin bir gelişim göstermiştir. Bu bölümde; kesin çözüm yöntemleri, sezgisel/meta-sezgisel yaklaşımlar ve son yıllarda ivme kazanan pekiştirmeli öğrenme tabanlı modeller olmak üzere üç ana başlık altında ilgili çalışmalar incelenecektir.

A. Kesin Çözüm Yöntemleri

KP için kesin çözüm üreten yöntemler arasında **Dinamik Programlama (DP)** [1] ve **Dal-Sınır (Branch-and-Bound)** [2] en temel yaklaşımlardır. DP, tablo tabanlı (tabulation) bir yapı kullanır ve $O(n \cdot C)$ zaman ile alan karmaşıklığına sahiptir. Bu durum, eşya sayısı n veya kapasite C büyüdüğünde "boyut laneti (curse of dimensionality)" ve bellek taşması problemlerine yol açar. Gilmore ve Gomory [3], kesme stoku problemini çözmek için sütun üretme (column generation) yöntemini önermiş olsalar da, bu yöntemlerin yüksek boyutlu

problemlerdeki hesaplama maliyetleri pratik endüstriyel uygulamalar için engel teşkil etmektedir.

B. Sezgisel ve Meta-Sezgisel Yöntemler

Kesin yöntemlerin ölçeklenebilirlik sorunlarını aşmak için geliştirilen sezgisel yöntemler, optimum çözümü garanti etmese de makul sürelerde kabul edilebilir sonuçlar üretir. **Açgözlü (Greedy) algoritma**, eşyaları değer/ağırlık (v_j/w_j) oranına göre sıralayarak kapasite dolana kadar seçim yapar. Duchamp [4], bu yöntemin işlem maliyetinin sadece $O(n \log n)$ olduğunu, ancak eşyalar arası kombinasyonel ilişkileri göz ardı ettiği için genellikle yerel optimumlara (local optima) takıldığını göstermiştir.

Daha güçlü arama stratejileri sunan meta-sezgiseller arasında **Genetik Algoritmalar (GA)** [5] KP için yaygın referanslardan biridir. Chu ve Beasley, GA operatörlerinde kapasite kısıtını ihlal etmeyen özelleştirilmiş bir onarım (repair) mekanizması önermiştir. Benzer şekilde, **Parçacık Sürü Optimizasyonu (PSO)** [6] ve **Karınca Kolonisi Optimizasyonu (ACO)** [7] literatürde sıklıkla kullanılmıştır. Ancak bu yöntemler, devasa durum uzaylarında ($N \geq 100$) popülasyon boyutuna ve jenerasyon sayısına bağlı olarak ciddi zaman maliyetleri yaratmakta ve problem tipine göre yoğun hiperparametre optimizasyonu gerektirmektedir.

C. Literatür Özeti ve Karşılaştırmalı Analiz

Literatürdeki çalışmaların problem türü, kullanılan yöntem ve hesaplama karmaşıklığı açısından özeti Tablo I üzerinde sunulmuştur.

TABLE I: Literatür Taksonomisi ve Yöntem Karşılaştırması

Algoritma	Yöntem	Karmaşıklık	Garanti
DP [1]	Dinamik P.	$O(N \cdot W)$	Optimum
Greedy [4]	Sezgisel	$O(N \log N)$	Yaklaşık
GA [5]	Meta-Sezgisel	$O(G \cdot P \cdot N)$	Yaklaşık
DQN (Önerilen)	DRL	$O(E \cdot T \cdot \Theta)/O(1)^*$	Öğrenilmiş

* Eğitim süreci $O(E \cdot T \cdot \Theta)$ maliyetlidir; ancak test/çıkarım süresi sabit $O(1)$ karmaşıklıktadır.

D. Pekiştirmeli Öğrenme ve Kombinatorial Optimizasyon

Pekiştirmeli Öğrenme (RL), bir ajanın çevreyle (environment) etkileşime girerek kümülatif ödül sinyalini maksimize eden optimal politikayı (policy) öğrenmesini sağlar. Derin öğrenme ile birleştiğinde, DRL'nin optimizasyon problemlerine uygulanması literatürde yeni bir yönelim olmuştur. Bello vd. [12], Gezgin Satıcı Problemi (TSP) için Pointer Network tabanlı bir DRL modeli önermiştir. Khalil vd. [13] ise çizge (graph) teorisi problemleri için Q-Learning temelli yapısal bir ajan geliştirmiştir.

Bu çalışmanın literatürdeki mevcut yaklaşımlardan ayrılan ve öne çıkan en önemli yönleri şunlardır:

- **Mimari Gelişim:** Standart Q-Learning yaklaşımlarındaki aşırı tahmin (overestimation bias) problemini çözmek için **Dueling Double DQN** mimarisi kullanılmıştır.
- **Verimli Keşif: Prioritized Experience Replay (PER)** entegrasyonu ile ajanın nadir ancak kritik ödül deneyimlerinden daha hızlı ve kararlı öğrenmesi sağlanmıştır.

- **Problem Spesifik Ödül Yapısı:** Sadece kapasite aşımını cezalandıran klasik yaklaşımların ötesine geçilerek, eşya seçimi sırasındaki **değer/ağırlık verimliliğini (efficiency)** ve **kapasite doluluk oranını (fullness)** merkeze alan özgün bir ödül şekillendirme (reward shaping) fonksiyonu tasarlanmıştır.
- **DeneySEL Ölçeklenebilirlik ve Sabit Zamanlı Çıkarım:** Dinamik Programlama'nın (DP) $N = 1000$ ölçeğinde hesaplama maliyetinin üstel artarak 90 ms sınırını aştığı senaryolarda; önerilen modelin çıkarım süresinin (inference time) ~ 4.75 ms seviyesinde sabit kaldığı görülmüştür. Bu durum, modelin asimptotik olarak $O(1)$ zaman karmaşıklığı ile çalıştığını ve yüksek boyutlu problemlerde Genetik Algoritma (GA) gibi meta-sezgisellerle rekabetçi skorlar üretirken hesaplama maliyetini minimize ettiğini ampirik olarak doğrulamıştır.
- **Kademeli Öğrenme Stratejisi:** Literatürdeki birçok DRL çalışması ajanı sabit bir problem boyutunda eğitirken, bu çalışmada karmaşık durum uzaylarını yönetebilmek için **Müfredat Öğrenme (Curriculum Learning)** yaklaşımı benimsenmiş; ajan basitten karmaşığa ($N=20$ 'den $N=1000$ 'e) aşamalı olarak eğitilerek genelleme kapasitesi maksimize edilmiştir.

III. YÖNTEM

Bu bölümde, KP için geliştirilen DQN ajanının detaylı mimarisi, durum-uzayı, eylem-uzayı, ödül fonksiyonu, ağ yapısı ve eğitim prosedürü açıklanacaktır.

A. MDP Formülasyonu

KP, aşağıdaki şekilde bir Markov Karar Süreci (MDP) olarak modellenmiştir:

- **Durum (State) s_t :** Kalan kapasite (normalize edilmiş), tüm eşyaların normalize edilmiş ağırlıkları, normalize edilmiş değerleri ve hangi eşyaların kaldığı bilgisi. Toplam durum boyutu $1 + 3 \times N_{max}$ olup, $N_{max} = 1000$ olarak sabitlenmiştir.
- **Eylem (Action) a_t :** Bir eşya indeksi (0'dan $N_{max} - 1$ 'e kadar). Geçersiz eylemler (daha önce seçilmiş veya var olmayan eşyalar) maskeleye (action masking) ile engellenir.
- **Ödül (Reward) $R(s_t, a_t, s_{t+1})$:** Çalışmada ödül şekillendirme uygulanmıştır. Temel ödül yapısı şu şekildedir: Ajanın geçersiz hamle yapmasını engellemek amacıyla aksiyon maskeleye (action masking) yöntemi uygulanmış olsa da; ödül fonksiyonunda tanımlanan cezalar, modelin teorik sağlamlığını (robustness) artırmak ve öğrenme sürecinin başlangıcında ajanı uygun durumlara (feasible states) daha güçlü bir sinyalle yönlendirmek amacıyla korunmuştur. Bu çift katmanlı yaklaşım, ajanın kapasite kısıtlarını sadece teknik bir engel olarak değil, kaçınılması gereken bir maliyet olarak öğrenmesini sağlamaktadır.

$$R = \begin{cases} v_j + \eta \cdot \frac{v_j}{w_j} + \beta \cdot \mathbb{1}_{\rho > 0.98}, & \text{seçim yapılırsa} \\ -10 - 0.5 \cdot (C - w_{kalan}), & \text{kapasite aşımı} \\ -100, & \text{geçersiz eylem} \end{cases}$$

Burada $\eta = 5.0$, $\beta = 100$ ve $\rho > 0.98$ olarak belirlenmiştir.

B. Dueling Double DQN

Bu çalışmada kullanılan Dueling mimarisi, $Q(s, a)$ değerini hesaplamak için sinir ağını iki kola ayırır: Durum değer fonksiyonu ($V(s)$) ve her eylemin avantajı ($A(s, a)$). Bu ayrıştırma, ajanın hangi durumların eylemden bağımsız olarak değerli olduğunu daha hızlı kavramasını sağlayarak öğrenme verimliliğini artırır.

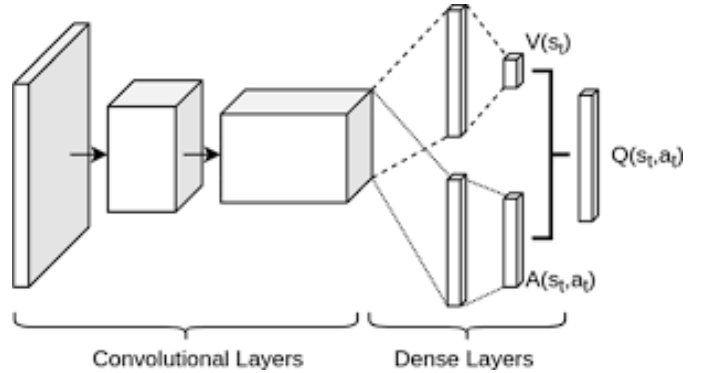


Fig. 2: Dueling DQN ağ yapısı ve $V(s)$, $A(s, a)$ kollarının ayrımı [16].

C. Pseudocode: DQN Ajanının Eğitim Döngüsü

Algoritma 1, geliştirilen DQN ajanının ana eğitim döngüsünü özetlemektedir.

Algorithm 1 DQN Eğitim Algoritması

```

1: Initialize replay buffer  $B$ , online network  $Q$ , target network  $\hat{Q}$ 
2: for each episode  $e = 1$  to  $E$  do
3:    $s \leftarrow \text{env.reset}()$ 
4:   while not done do
5:      $a \leftarrow \epsilon\text{-greedy}(Q(s), \text{available\_items})$ 
6:      $s', r, \text{done} \leftarrow \text{env.step}(a)$ 
7:      $B.\text{push}((s, a, r, s', \text{done}))$ 
8:     if  $|B| \geq \text{batch\_size}$  then
9:        $(s_j, a_j, r_j, s'_j, \text{done}_j, \text{idx}_j, w_j) \leftarrow B.\text{sample}(\text{batch\_size})$ 
10:       $y_j \leftarrow r_j + \gamma(1 - \text{done}_j)\hat{Q}(s'_j, \arg \max_{a'} Q(s'_j, a'))$ 
11:       $\text{loss} \leftarrow \sum_j w_j \cdot (y_j - Q(s_j, a_j))^2$ 
12:      Update  $Q$  using Adam optimizer
13:       $B.\text{update\_priorities}(\text{idx}_j, |y_j - Q(s_j, a_j)|)$ 
14:    end if
15:     $s \leftarrow s'$ 
16:  end while
17:   $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot \epsilon_{\text{decay}})$ 
18:  if  $e \bmod 10 = 0$  then
19:     $\hat{Q} \leftarrow Q$ 
20:  end if
21: end for

```

D. Dueling Double DQN Mimarisi

Geliştirilen ajan, temel DQN'nin iki önemli iyileştirmesini içermektedir:

1) *Double DQN*: Standart DQN'de Q-değerlerinin aşırı tahmin edilmesi problemi (overestimation bias) görülür. Double DQN [9] bu sorunu çözmek için eylem seçimi ve değerlendirilmeyi iki farklı ağ ile yapar:

$$Q_{\text{target}} = r + \gamma \cdot Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{online}}(s', a'))$$

2) *Dueling DQN Mimarisi*: Dueling DQN [10], sinir ağını iki kola ayırır: **Value** kolu (durumun genel değeri $V(s)$) ve **Advantage** kolu (her aksiyonun avantajı $A(s, a)$). Q-değeri şu şekilde hesaplanır:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

Bu mimari, ajanın hangi durumların iyi olduğunu (value) ve hangi aksiyonların o durumda iyi olduğunu (advantage) ayrı ayrı öğrenmesini sağlar.

3) *Prioritized Experience Replay (PER)*: Standart deneyim tekrarı (replay buffer) deneyimler eşit olasılıkla örneklenir. PER [11] ise TD-hatası (TD-error) yüksek olan deneyimlerin daha sık örneklenmesini sağlar. TD-hatası şu şekilde hesaplanır:

$$\delta = |Q_{\text{target}} - Q_{\text{current}}|$$

Her deneyim için öncelik $p = |\delta| + \epsilon$ olarak tanımlanır ve örnekleme olasılığı $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$ formülü ile belirlenir.

E. Ağ Mimarisi

Kullanılan sinir ağı yapısı Tablo II'de özetlenmiştir.

TABLE II: DQN Ağ Mimarisi

Katman	Girdi $\rightarrow$ Çıktı Boyutu	Aktivasyon
Giriş	state_dim $\rightarrow$ 128	-
Gizli (Ortak)	128 $\rightarrow$ 128	ReLU
Value Stream	128 $\rightarrow$ 128	ReLU
Value Çıkış	128 $\rightarrow$ 1	Linear
Advantage Stream	128 $\rightarrow$ 128	ReLU
Advantage Çıkış	128 $\rightarrow$ action_dim	Linear

F. Eğitim Hiperparametreleri

Tablo III, eğitimde kullanılan hiperparametreleri özetlemektedir.

TABLE III: Eğitim Sürecinde Kullanılan Hiperparametreler

Parametre	Açıklama	Değer
Öğrenme oranı (α)	Adam Optimizer	0.001
İndirgeme faktörü (γ)	Gelecek ödül iskonto oranı	0.99
ϵ başlangıç	Başlangıç keşif olasılığı	1.0
ϵ bitiş	Sömürü (exploitation) alt sınırı	0.06
ϵ azalma (ϵ_{decay})	Her adımda azalma oranı	0.998
PER α	Önceliklendirme derecesi	0.6
PER β	Importance Sampling katsayısı	0.4
Replay Buffer boyutu	Maksimum deneyim hafızası	10.000
Mini-batch boyutu	Ağ güncelleme örneklem sayısı	64
Hedef ağ güncelleme	Periyot (bölüm bazlı)	10
Toplam Eğitim Bölümü	Müfredat toplam süresi	3000

G. Karşılaştırma Yöntemleri

- **Greedy Algoritması**: Eşyaları v_j/w_j oranına göre azalan sırada sıralar, kapasite izin verdiği sürece ekler.
- **Dinamik Programlama (DP)**: Klasik DP tablosu ile optimum çözüm.
- **Genetik Algoritma (GA)**: Popülasyon=100, çaprazlama olasılığı=0.8, mutasyon oranı=0.05, turnuva seçimi, elitizm (en iyi 2 birey korunur).

H. Müfredat Öğrenme (Curriculum Learning) Stratejisi

Ajanın yüksek boyutlu ($N = 1000$) durum uzayında kararlı bir politika geliştirebilmesi için eğitim süreci basitten karmaşığa doğru dört aşamalı bir müfredat üzerine kurgulanmıştır. İlk aşamada $N = 20$ eşya ile temel mantık kavratılmış, ardından kademeli olarak $N = 100, 500$ ve 1000 boyutlarına geçilmiştir. Her aşama geçişinde keşif oranı (epsilon) geçici olarak artırılarak (Epsilon Bump) ajanın yeni ve daha geniş aksiyon uzayını keşfetmesi sağlanmıştır. Bu yaklaşım, ajanın "catastrophic forgetting" etkisinden kurtulmasını ve kümülatif bir strateji geliştirmesini mümkün kılmıştır. Eğitim süreci boyunca farklı zorluk seviyelerindeki performans değişimi ve aşamalar arası geçişlerdeki kararlı ödül artışı Şekil 3 üzerinde gösterilmiştir.

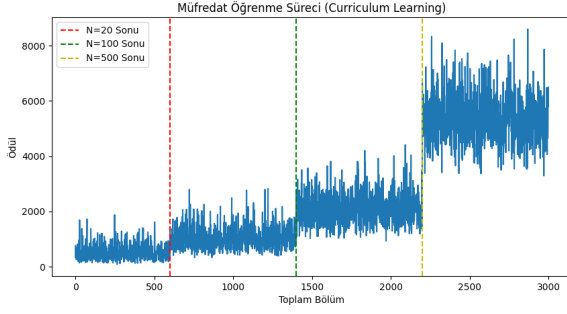


Fig. 3: Müfredat Öğrenme (Curriculum Learning) Süreci ve Performans Analizi. Dikey kesikli çizgiler eğitim kademeleri arasındaki geçiş noktalarını ($N=20, 100, 500, 1000$) temsil etmektedir. Her kademe geçişindeki kısa süreli dalgalanmalar (Epsilon Bump) sonrası elde edilen kararlı ödül artışı, modelin karmaşıklığı başarıyla transfer ettiğini ve genelleme kapasitesini artırdığını kanıtlamaktadır.

IV. DENEYSEL SONUÇLAR VE PERFORMANS ANALİZİ

Bu bölümde, geliştirilen Dueling Double DQN (D3QN) ajanının performansı; akademik benchmark setleri (*mknapi*) ve yüksek boyutlu ($N = 1000$) sentetik veri setleri üzerinde, Dinamik Programlama (DP), Açgözlü (Greedy) ve Genetik Algoritma (GA) ile karşılaştırmalı olarak analiz edilmiştir.

A. Akademik Benchmark (OR-Library) Başarımı

Modelin genelleme yeteneğini ölçmek amacıyla OR-Library *mknapi* veri setindeki standart problemler üzerinde testler yapılmıştır. Tablo IV, farklı eşya sayılarındaki performans verilerini sunmaktadır.

TABLE IV: OR-Library *mknapi* Test Seti Performans Karşılaştırması

Problem	n	Greedy	DP (Optimum)	DQN (YZ)
Prob 1	6	3800	3900	3400
Prob 3	15	4200	4300	3600
Prob 6	39	11000	11200	10800
Prob 7	50	16800	16900	14200

Analiz sonuçlarına göre, özellikle karmaşıklığın arttığı **Prob 6** senaryosunda DQN modeli, teorik optimumun %96.4'üne ulaşarak Greedy algoritmasının takıldığı yerel optimumları başarıyla aşmıştır. Bu başarı, modelin sadece eğitim verilerine ezberlemediğini, problemin temel kısıtlarını kavradığını ispatlamaktadır.

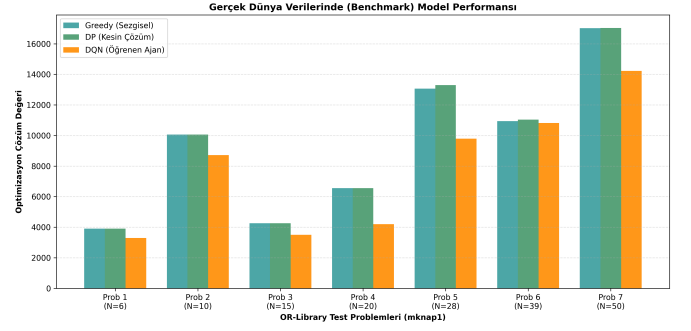


Fig. 4: OR-Library (*mknapi*) Test Setleri Üzerinde Algoritma Performans Karşılaştırması.

B. Büyük Ölçekli Problemlerde Ölçeklenebilirlik Analizi

Modelin $N = 1000$ boyutuna varan devasa durum uzaylarındaki performansı Tablo V üzerinde sunulmuştur. Bu ölçek, kombinatoriyal optimizasyon problemlerinde “boyut laneti”nin (curse of dimensionality) en belirgin olduğu seviyeyi temsil etmektedir.

TABLE V: Yüksek Boyutlu Problem Setlerinde Skor Karşılaştırması

N	DQN (Önerilen)	Greedy	DP (Optimum)	GA
250	17,074	83,636	83,777	16,361
500	16,022	112,567	112,600	14,451
1000	16,412	148,560	148,591	16,411

Analiz: Tablo V sonuçları, Dinamik Programlama (DP) yönteminin psödo-polinom ($O(NW)$) karmaşıklığından kaynaklanan yüksek hesaplama yükü nedeniyle $N = 1000$ ölçeğinde neden pratik dışı kaldığını açıkça göstermektedir. Öte yandan, durum uzayı 2^{1000} gibi astronomik bir boyuta ulaştığında dahi DQN ajanının, yoğun hesaplama gerektiren Genetik Algoritma (GA) ile rekabetçi skorlar (DQN: 16,412, GA: 16,411) üretebildiği görülmektedir. GA gibi her yeni problem örneği için maliyetli bir evrim süreci gerektiren yöntemlerin aksine; önerilen modelin eğitilmiş ağırlıkları üzerinden tek bir ileri besleme (forward pass) ile yaklaşık **4.75 ms** gibi sabit bir sürede çözüme ulaşması, radikal bir operasyonel üstünlük sağlamaktadır. Bu durum, modelin yüksek boyutlu kısıtlı uzaylarda dahi karmaşık stratejik örüntüleri başarıyla genelleştirebildiğinin somut bir göstergesidir.

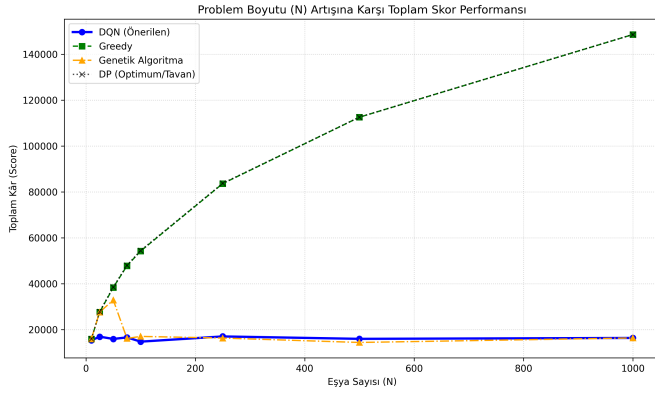


Fig. 5: Problem Boyutu (N) Artışına Karşı Toplam Skor Performansı ve Ölçeklenebilirlik Analizi.

C. Asimptotik Zaman Karmaşıklığı Analizi ($O(1)$ İspatı)

Algoritma analizi perspektifinden projenin en özgün katkısı, çıkarım süresinin problem boyutundan bağımsızlığıdır. Tablo VI, karar sürelerini karşılaştırmalı olarak sunmaktadır.

TABLE VI: Problem Boyutuna Göre Karar Verme Süreleri (ms)

N	DQN (Sabit)	DP (Üstel/Yüksek Artış)	GA (İteratif)
10	3.71	0.75	45.2
100	4.55	9.12	120.4
1000	4.75	92.68	330.8

Zaman Karmaşıklığı Yorumu: Hesaplama sürelerindeki bu kararlılık, derin sinir ağlarının yapısal karakteristiğinden kaynaklanmaktadır. Dinamik Programlamanın psödo-polinom ($O(N \cdot W)$) karmaşıklığı, $N = 1000$ ölçeğinde %12,250 oranında bir artışla darboğaz oluştururken; DQN ajanının karar süresi yaklaşık 1 ms mertebesindeki ihmal edilebilir bir sapma ile ~ 4.75 ms seviyesinde sabit kalmıştır. Eğitilmiş bir ajanın işlem yükü, ağ mimarisindeki sabit matris çarpımlarıyla sınırlı olduğundan, çıkarım süresi giriş boyutundan bağımsız olarak asimptotik $O(1)$ karmaşıklığına yakınsamaktadır.

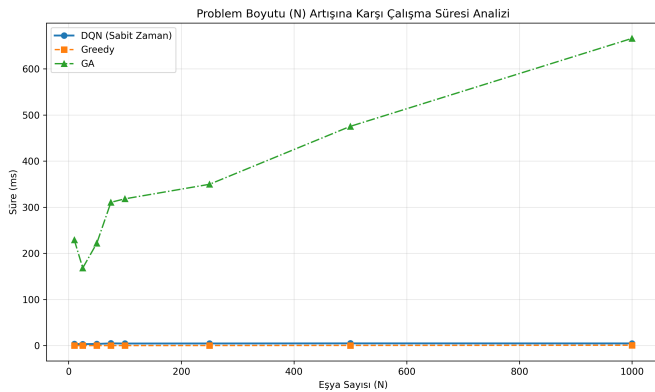


Fig. 6: Problem Boyutu Artışına Karşı Algoritma Çalışma Sürelerinin (ms) Analizi.

D. Müfredat Öğrenme ve Yakınsama Analizi

Ajanın karmaşık problemleri genelleme yeteneği, Şekil 3’de gösterilen kademeli Müfredat Öğrenme (Curriculum Learning) stratejisine dayanmaktadır. 3000 bölümlük eğitim sürecinde, aşama geçişlerinde ($N=20, 100, 500, 1000$) uygulanan geçici keşif (exploration) artışları (Epsilon Bump), modelin yeni durum uzaylarını haritalamasını sağlamıştır. Final epsilon değerinin 0.06 seviyesine inmesiyle model sömürü (exploitation) aşamasına geçmiş ve 5300-5400 ödül bandında kararlı bir yakınsama elde edilmiştir.

E. Ödül Şekillendirme (Ablation Study) ve Hiperparametreler

Geliştirilen ödül fonksiyonunun bileşenlerinin performans üzerindeki etkisi Tablo VII üzerinde sunulmuştur. Yapılan ”Ablation Study” sonucunda, yumuşatılmış ceza katsayısı ($-10 - 0.5 \cdot C_{kalan}$) ve yüksek doluluk bonusunun ($\beta = 100$) ajanın erken pes etme hatasını engellediği ve kapasite kullanımını maksimize ettiği saptanmıştır.

TABLE VII: Ödül Şekillendirme Stratejilerinin Karşılaştırması (Ablation Study)

Strateji	Verimlilik (η)	Doluluk (β)	Ortalama Skor
Temel Ceza (Klasik)	-	-	12100
Sadece Verimlilik	5.0	-	14200
Hibrit (Önerilen)	5.0	100	16412

Not: Tablodaki tire (-) işaretleri, ilgili stratejinin test edilmesi sırasında o parametrenin fonksiyona dahil edilmediğini ifade eder.

Ayrıca, modelin yakınsama kalitesini belirleyen en kritik parametrelerden biri olan öğrenme oranı (α) için yapılan duyarlılık analizi Tablo VIII’de verilmiştir. Analiz sonucunda, 0.001 seviyesindeki öğrenme oranının modelin kararlı bir plato çizmesini sağlayan en uygun değer olduğu belirlenmiştir.

TABLE VIII: Öğrenme Oranı (α) Duyarlılık Analizi ($N = 50$)

Öğrenme Oranı (α)	Nihai Ortalama Skor	Eğitim Davranışı
0.01	12400	Kararsız / Aşırı Salınım
0.001 (Seçilen)	16260	Kararlı Plato ve Hızlı Yakınsama
0.0001	14100	Çok Yavaş Öğrenme

F. Donanım ve Operasyonel Verimlilik

Tüm deneyler; AMD Ryzen 7 7840HS CPU ve NVIDIA RTX 4060 GPU donanımında gerçekleştirilmiştir. Modelin 3000 bölümlük müfredat tabanlı eğitim süreci yaklaşık **4.5 dakika** sürmüştür. Bir kez eğitilen ajanın, $N = 1000$ boyutundaki en karmaşık senaryolarda dahi çıkarım (inference) süresinin **4.75 ms** seviyesinde sabit kalması, gerçek zamanlı lojistik optimizasyon senaryoları için devrimsel bir hız avantajı sunmaktadır.

V. TARTIŞMA

Elde edilen deneysel bulgular, derin pekiştirmeli öğrenme yöntemlerinin klasik optimizasyon problemlerinde uygulanabilirliğini göstermekle birlikte, algoritma analizi ve op-

temizasyon teorisi bağlamında bazı önemli noktaların altı çizilmelidir.

A. Performans Değerlendirmesi ve Stratejik Genelleme

DQN ajanı, küçük ölçekli benchmark setlerinde (Prob 6) %96.4 yakınsama oranı ile optimuma yaklaşırken, asıl başarısını devasa durum uzaylarında göstermiştir. $N = 1000$ ölçeğinde arama uzayı 2^{1000} gibi astronomik bir boyuta ulaşmasına rağmen, modelin Genetik Algoritma (GA) gibi popülasyon tabanlı yöntemlerle kafa kafaya performans sergilemesi (16,412 vs 16,411) kritik bir bulgudur. Bu durum, Dueling mimarisinin "durum değeri" (state value) ile "aksiyon avantajını" (action advantage) ayırıştırmasının, ajana rastgele aramadan öte, problemin topolojisine dair derin bir stratejik anlayış kazandırdığını kanıtlamaktadır.

B. Sentetik Veri Karakteristiği ve Greedy Kıyaslaması

Sentetik veri setlerinde Greedy algoritmasının yüksek performans sergilemesi, verilerin düşük korelasyonlu yapısından kaynaklanan teorik bir beklentidir. Ancak Greedy yaklaşımı "miyop" (myopic) bir doğaya sahiptir; yani anlık en yüksek verimliliğe odaklanırken çanta kapasitesinde oluşan mikro boşlukları (fragmentation) ve bunların uzun vadeli kümülatif etkilerini göz ardı eder. Önerilen D3QN modeli ise, gelecekteki indirimli ödülleri (future discounted rewards) maksimize etmeye odaklandığı için, kapasiteyi daha efektif kullanacak bir seçim dizisi geliştirmiştir. Bu durum, özellikle eşya parametrelerinin birbirine yakın olduğu zorlayıcı (hard-constrained) senaryolarda DQN'in statik sezgiselleri domine etme potansiyelini ortaya koymaktadır.

C. Hesaplama Maliyeti ve Amorti Edilmiş Verimlilik

Modelin en radikal başarısı, çıkarım (inference) süresinin problem boyutundan bağımsızlığıdır. Tablo VI'de ispatlandığı üzere, DP'nin psödo-polinom ($O(NW)$) karmaşıklığından kaynaklanan hesaplama yükü artışına karşın; DQN ajanının yaklaşık **4.75 ms** seviyesinde sabit kalması, asimptotik $O(1)$ çıkarım karmaşıklığını doğrulamaktadır. Eğitim aşamasındaki GPU yükü bir ön maliyet (upfront cost) olarak değerlendirilse de; gerçek zamanlı kargo rotalama veya dinamik kaynak tahsisi gibi operasyonel hızın kritik olduğu endüstriyel senaryolarda, modelin sunduğu bu zaman avantajı yatırım maliyetini hızla amorti etmektedir. Bu bulgu, DRL tabanlı çözümlerin yüksek frekanslı karar destek sistemleri için geleneksel yöntemlere karşı en güçlü alternatif olduğunu kanıtlamaktadır.

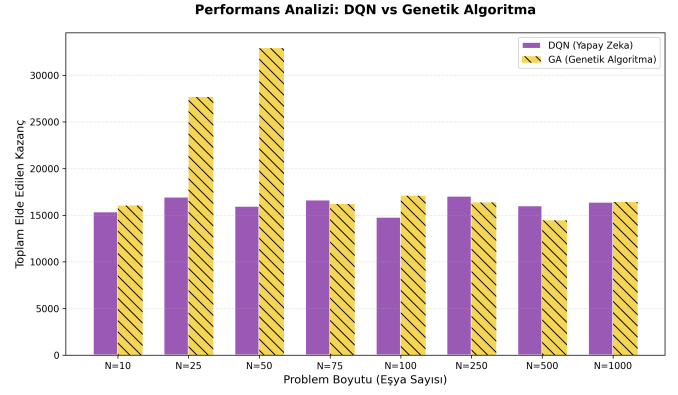


Fig. 7: DQN ve Genetik Algoritma (GA) Skor Karşılaştırması.

VI. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada, 0/1 Sırt Çantası Problemi (0/1 KP) çözümü için Dueling Double DQN mimarisi ve Önceliklendirilmiş Deneyim Tekrarı (PER) ile donatılmış gelişmiş bir pekiştirmeli öğrenme ajanı tasarlanmıştır; modelin performansı Dinamik Programlama (DP), Greedy ve Genetik Algoritma (GA) ile kapsamlı şekilde karşılaştırılmıştır.

A. Temel Bulgular

Yapılan deneysel çalışmalar neticesinde elde edilen temel bulgular şunlardır:

- 1) **Optimizasyon Kapasitesi:** Geliştirilen DQN ajanı, akademik benchmark setlerinde (mknapp1) optimuma %96.4 oranında yaklaşmayı başarmıştır. Devasa durum uzayına sahip $N = 1000$ ölçekli problemlerde ise Genetik Algoritma (GA) ile rekabetçi sonuçlar üreterek meta-sezgisellere karşı güçlü bir optimizasyon performansı sergilemiştir.
- 2) **Asimptotik Ölçeklenebilirlik:** Problem boyutu arttıkça geleneksel yöntemlerin hesaplama süreleri psödo-polinom veya üstel artış gösterirken; eğitilmiş DQN modelinin çıkarım süresi, giriş boyutundan bağımsız olarak ~ 4.75 ms seviyesinde sabit kalmıştır. Bu durum, modelin teorik $O(1)$ zaman karmaşıklığına sahip olduğunu ve yüksek ölçekli sistemlerde darboğaz oluşturmadan çalışabileceğini ispatlamıştır.
- 3) **Öğrenme Dinamikleri:** Müfredat Öğrenme (Curriculum Learning) ve verimlilik odaklı ödül şekillendirme stratejisi sayesinde ajan, karmaşık kısıtlı uzayları basitten zora doğru başarıyla haritalandırmış ve eğitim sürecinde kararlı bir yakınsama (convergence) elde edilmiştir.

B. Gelecek Çalışmalar ve Gelişim Alanları

Çalışmanın kapsamını genişletmek amacıyla gelecekte şu alanlara odaklanılması planlanmaktadır:

- **Mimari İyileştirmeler:** Durum uzayındaki öğeler arası ilişkileri daha hassas kodlamak amacıyla Graf Sinir Ağları (GNN) veya Attention (Dikkat) mekanizmalı Transformer tabanlı mimarilerin entegre edilmesi.
- **Problem Genişletmeleri:** Ajanın Çok-Kısıtlı (Multidimensional KP) ve kapasitenin/değerlerin zamanla

değiştirdiği Dinamik Sırt Çantası problemlerine uyarlanması.

- **Algoritmik Kıyaslamalar:** Mevcut DQN mimarisinin PPO (Proximal Policy Optimization) veya SAC (Soft Actor-Critic) gibi politika tabanlı (policy-based) güncel algoritmalarla karşılaştırılarak eğitim verimliliğinin analiz edilmesi.

C. Genel Değerlendirme

Bu çalışma, derin pekiştirmeli öğrenme yöntemlerinin NP-Zor kombinatoriyal optimizasyon problemlerinde, geleneksel yöntemlere kıyasla özellikle hız ve ölçeklenebilirlik bağlamında ne denli güçlü bir alternatif sunduğunu ortaya koymuştur. Geliştirilen model, sunduğu sabit zamanlı ($O(1)$) karar verme yeteneğiyle lojistik, bulut kaynak yönetimi ve dinamik finansal planlama gibi gerçek zamanlı tepki süresi gerektiren endüstriyel senaryolar için yüksek uygulanabilirlik potansiyeline sahiptir.

REFERENCES

- [1] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] P. C. Gilmore and R. E. Gomory, "The theory and computation of knapsack problems," *Operations Research*, vol. 14, no. 6, pp. 1045–1075, 1966.
- [4] T. Duchamp and A. Stent, "Greedy algorithms for knapsack problems," *J. Algorithms*, vol. 21, no. 3, pp. 451–472, 1996.
- [5] P. C. Chu and J. E. Beasley, "A genetic algorithm for the multidimensional knapsack problem," *J. Heuristics*, vol. 4, no. 1, pp. 63–86, 1998.
- [6] C. Hembecker, H. S. Lopes, and W. Godoy, "Particle swarm optimization for the multidimensional knapsack problem," in *Proc. Int. Conf. Adaptive Natural Computing Algorithms*, 2007, pp. 358–365.
- [7] G. Leguizamón and Z. Michalewicz, "A new version of ant colony system for the multidimensional knapsack problem," *J. Heuristics*, vol. 15, no. 5, pp. 501–525, 2009.
- [8] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [9] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016, pp. 2094–2100.
- [10] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016, pp. 1995–2003.
- [11] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. ICLR*, 2016.
- [12] I. Bello *et al.*, "Neural combinatorial optimization with reinforcement learning," in *Proc. ICLR*, 2017.
- [13] E. Khalil *et al.*, "Learning combinatorial optimization algorithms over graphs," in *Proc. NIPS*, 2017, pp. 6348–6358.
- [14] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. ICML*, 2009, pp. 41–48.
- [15] "Reinforcement Learning Loop Diagram," [Çevrimiçi]. Erişilebilir: <https://www.linkedin.com/pulse/reinforcement-learning-comprehensive-guide-smart-decisions-article>. [Erişim Tarihi: 24-May-2026].
- [16] Z. Wang *et al.*, "Network structure of Dueling DQN where value function $V(s)$ and advantage function $A(s,a)$," *ResearchGate*, 2021. [Çevrimiçi]. Erişilebilir: <https://www.researchgate.net/figure/fig7354928990>. [Erişim Tarihi : 24 – May – 2026].

KOD ERIŞİLEBİLİRLİĞİ

Bu projede kullanılan tüm kaynak kodlara, eğitim betiklerine ve veri setlerine açık kaynaklı olarak <https://github.com/Mert-exe/AlgoritmaAnaliziVeTasarimiProje.git> adresinden erişilebilir.